

Combinatorics in DSA

Absolutely. **Combinatorics is one of the most useful mathematical ideas in DSA**, especially when a problem asks:

- "How many ways...?"
- "How many possible arrangements...?"
- "How many subsets...?"
- "How many paths...?"
- "Choose k elements..."
- "Count the number of ways..."
- "What is the probability...?"

The main idea is actually simple:

Combinatorics = counting how many possible configurations/choices exist without explicitly generating all of them.

1. What exactly is Combinatorics?

Suppose you have:

```
[1, 2, 3]
```

How many ways can you choose elements?

You could choose:

```
{ }
{1}
{2}
{3}
{1,2}
{1,3}
{2,3}
{1,2,3}
```

There are **8 subsets**.

Instead of generating them, combinatorics tells us:

$$2^n$$

For $n = 3$:

$$2^3 = 8$$

Why?

For every element, we have exactly **2 choices**:

```
Take it
Don't take it
```

So:

```
element 1 → 2 choices
element 2 → 2 choices
element 3 → 2 choices
```

Therefore:

$$2 \times 2 \times 2 = 2^3$$

This is the basic thinking behind combinatorics.

2. Why is Combinatorics important in DSA?

A lot of DSA problems are secretly asking:

How many possibilities are there?

For example:

Problem

You have n elements. Find the number of subsets.

A brute-force approach:

```
C++
generate_subset 1
generate_subset 2
generate_subset 3
...
```

There are:

$$2^n$$

subsets.

For $n = 50$:

$$2^{50} \approx 1.12 \times 10^{15}$$

Impossible to enumerate.

But if the question only asks for the **count**, combinatorics gives us the answer immediately.

3. The most important concepts for DSA

You should know these particularly well:

1. Fundamental Counting Principle
2. Permutations
3. Combinations
4. Factorials
5. Subsets
6. Stars and Bars
7. Inclusion-Exclusion
8. Pigeonhole Principle
9. Catalan Numbers
10. Modular Combinatorics

Let's understand them one by one.

4. Fundamental Counting Principle

This is probably the **most important concept**.

Suppose you have:

```
3 shirts
2 pants
```

How many outfits can you make?

For every shirt, you have 2 choices of pants.

Therefore:

$$3 \times 2 = 6$$

So:

Total ways = 3×2

DSA example

Suppose a password contains:

```
2 digits
```

Each digit can be:

```
0 - 9
```

So each position has 10 possibilities.

Therefore:

$$10 \times 10 = 100$$

For n digits:

$$10^n$$

This idea appears everywhere in DSA.

5. Factorial

Factorial is:

$$n! = n \times (n - 1) \times (n - 2) \times \dots \times 1$$

Examples:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

$$3! = 6$$

$$1! = 1$$

$$0! = 1$$

6. Permutation

Permutation means:

Arrangement matters.

Suppose:

A B C

How many ways can we arrange them?

ABC
ACB
BAC
BCA
CAB
CBA

Answer:

$$3! = 6$$

Formula

Number of ways to arrange r objects from n objects:

$${}^n P_r = \frac{n!}{(n - r)!}$$

Example:

You have:

5 people

Choose and arrange:

3 people

Then:

$$\begin{aligned} {}^5 P_3 &= \frac{5!}{(5 - 3)!} \\ &= \frac{5!}{2!} \\ &= 5 \times 4 \times 3 \end{aligned}$$

$$= 60$$

7. Combination

Combination means:

Selection matters, but order does NOT matter.

Suppose:

A B C

Choose 2.

Possible choices:

AB
AC
BC

We don't count:

AB
BA

as different because we are simply choosing the same two elements.

Therefore:

$${}^nC_r = \frac{n!}{r!(n-r)!}$$

For example:

$${}^5C_2 = \frac{5!}{2!3!} = 10$$

8. Permutation vs Combination

This is **very important in DSA**.

Ask yourself:

Does order matter?

Order matters → Permutation

Example:

ABC
ACB

These are different.

Use:

$$P(n, r)$$

Order doesn't matter → Combination

Example:

AB
BA

These represent the same selection.

Use:

$$C(n, r)$$

Easy memory trick

Think:

Combination

Choose

Permutation

Arrange

9. Subsets

Suppose:

```
[1, 2, 3]
```

Each element has two choices:

```
include
exclude
```

Therefore:

$$2^n$$

subsets.

For $n = 4$:

$$2^4 = 16$$

This is extremely common in:

- Bitmasking
- Backtracking
- DP
- Subset problems
- Meet-in-the-middle

10. Bitmask + Combinatorics

This is a very common DSA connection.

Suppose:

```
nums = [A, B, C]
```

Represent each subset using binary.

```
000 -> {}
001 -> {C}
010 -> {B}
011 -> {B, C}
100 -> {A}
101 -> {A, C}
110 -> {A, B}
111 -> {A, B, C}
```

There are:

$$2^3$$

masks.

General:

$$2^n$$

This is why subset problems frequently use:

```
C++
for (int mask = 0; mask < (1 << n); mask++) {
}
```

Because:

$$1 \ll n = 2^n$$

11. Choosing exactly K elements

Suppose:

$$n = 5$$

and you want to choose exactly:

$$k = 2$$

elements.

Answer:

$$C(5, 2) = 10$$

For example:

```
{1, 2}
{1, 3}
{1, 4}
{1, 5}
{2, 3}
{2, 4}
{2, 5}
{3, 4}
{3, 5}
{4, 5}
```

This appears constantly in DSA.

12. Pascal's Triangle

Combination has an important relationship:

$$C(n, k) = C(n - 1, k - 1) + C(n - 1, k)$$

For example:

$$\begin{aligned} C(5, 2) &= C(4, 1) + C(4, 2) \\ &= 4 + 6 = 10 \end{aligned}$$

This creates Pascal's Triangle:

```
      1
     1 1
    1 2 1
   1 3 3 1
  1 4 6 4 1
 1 5 10 10 5 1
```

This is useful in:

- DP
- Combination calculations
- Counting paths
- Mathematical problems

13. Counting Grid Paths

This is a **very important DSA application**.

Suppose you need to go from:

```
Start → → →
      ↓
      ↓
      ↓
      ↓
      End
```

Suppose you need:

3 moves right
2 moves down

Total moves:

$$3 + 2 = 5$$

Among these 5 moves, choose where the 2 down moves occur.

Therefore:

$$\begin{aligned} C(5, 2) \\ = 10 \end{aligned}$$

So there are:

$$\boxed{10}$$

different paths.

General formula

If you need:

R right moves
D down moves

then:

$$\boxed{C(R + D, R)}$$

or equivalently:

$$\boxed{C(R + D, D)}$$

This appears in:

- Grid DP
- Number of paths
- Combinatorial DP
- Lattice problems

14. Example: Unique Paths

LeetCode has the classic problem:

A robot is at the top-left of an $m \times n$ grid and can only move right or down. Find the number of unique paths.

Instead of DP, mathematically:

Number of moves:

$$(m - 1) + (n - 1)$$

Therefore:

$$\boxed{C(m + n - 2, m - 1)}$$

For:

m = 3
n = 7

we get:

$$C(8, 2) = 28$$

So:

answer = 28

This is a beautiful example of combinatorics being used directly in DSA.

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

15. Inclusion-Exclusion

Sometimes you want to count:

A OR B

But some elements belong to both.

If:

```
A = 10
B = 20
A ∩ B = 5
```

Then:

$$|A \cup B| = |A| + |B| - |A \cap B|$$

Therefore:

$$10 + 20 - 5 = 25$$

Why subtract?

Because the intersection was counted twice.

DSA uses

Inclusion-exclusion appears in:

- Counting problems
- Number theory
- Divisibility problems
- Set problems
- Counting strings
- Probability
- Combinatorial DP

16. Pigeonhole Principle

Very simple idea:

If you put more than n objects into n boxes, at least one box must contain more than one object.

Example:

```
13 people
12 months
```

At least two people must have birthdays in the same month.

Why?

Because:

```
13 people > 12 months
```

DSA application

Suppose you have:

```
n + 1 numbers
```

and every number belongs to:

```
n possible categories
```

Then at least two numbers must belong to the same category.

This is useful for:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

- Duplicate detection
- Hashing ideas
- Number theory
- Array problems
- Proof-based problems

17. Stars and Bars

This is extremely useful for advanced counting problems.

Suppose:

$$x_1 + x_2 + x_3 = 5$$

where:

$$x_1, x_2, x_3 \geq 0$$

How many solutions?

Formula:

$$C(n + k - 1, k - 1)$$

Here:

$$\begin{aligned} n &= 5 \\ k &= 3 \end{aligned}$$

Therefore:

$$\begin{aligned} C(5 + 3 - 1, 3 - 1) \\ = C(7, 2) \\ = 21 \end{aligned}$$

So there are:

$$21$$

solutions.

18. Catalan Numbers

This is another very important combinatorial sequence in DSA.

Catalan numbers:

```
1
1
2
5
14
42
132
...
```

Formula:

$$C_n = \frac{1}{n+1} C(2n, n)$$

Catalan numbers appear in problems involving:

- Valid parentheses
- Binary search trees
- Binary trees
- Stack permutations
- Polygon triangulation
- Divide-and-conquer structures

Example: Valid Parentheses

For:

()

There is:

1

valid arrangement.

For:

(())

and:

()()

There are:

2

valid arrangements.

For 3 pairs:

5

valid arrangements.

So:

1, 2, 5, 14, 42...

These are Catalan numbers.

19. Combinatorics + Dynamic Programming

Combinatorics and DP often work together.

For example:

How many ways can I reach position n?

You might derive:

$$dp[i] = dp[i - 1] + dp[i - 2]$$

That's counting different possibilities.

Similarly:

How many ways to make sum X?

or:

How many ways to form a string?

or:

How many paths exist?

are often **counting problems**, where DP stores the number of ways.

20. Combinatorics + Backtracking

Suppose:

n = 10

and you want all subsets.

Number of subsets:

$$2^{10} = 1024$$

Backtracking can generate them.

But if:

`n = 50`

then:

$$2^{50}$$

is enormous.

Combinatorics lets you understand **how large the search space is before writing the algorithm**.

This is extremely important.

21. Combinatorics + Time Complexity

This is one of the biggest practical uses.

Suppose you need to check every pair:

$$C(n, 2)$$

which is:

$$\frac{n(n-1)}{2}$$

Therefore:

$$O(n^2)$$

For every triplet:

$$C(n, 3)$$

which is:

$$\frac{n(n-1)(n-2)}{6}$$

Therefore:

$$O(n^3)$$

So combinatorics helps you understand why:

```
pairs → O(n²)
triplets → O(n³)
subsets → O(2ⁿ)
permutations → O(n!)
```

This is **very important for competitive programming**.

22. Search Space Size

You should memorize this rough hierarchy:

```
O(1)
O(log n)
O(n)
O(n log n)
O(n²)
O(n³)
O(2ⁿ)
O(n!)
```

For example:

Choose every pair

$$C(n, 2) \approx n^2$$

Choose every triplet

$$C(n, 3) \approx n^3$$

Generate every subset

$$2^n$$

Generate every permutation

$$n!$$

This tells you immediately whether brute force is possible.

23. Combinatorics + Modular Arithmetic

This becomes extremely important in Codeforces/LeetCode.

You might see:

Return the answer modulo $10^9 + 7$.

Suppose:

$$C(n, k)$$

is enormous.

For example:

$$C(10^5, 50000)$$

The actual number is ridiculously large.

You can't store it in:

```
C++
int
```

or even:

```
C++
long long
```

So we calculate:

$$C(n, k) \mod (10^9 + 7)$$

This introduces:

- Modular inverse
- Fermat's Little Theorem
- Factorial precomputation
- Inverse factorial
- Fast exponentiation

24. Common DSA Formula

For large n under a prime modulus:

$$C(n, k) = \frac{n!}{k!(n-k)!}$$

Modulo MOD :

$$C(n, k) = fact[n] \times invFact[k] \times invFact[n-k] \mod MOD$$

where:

```
fact[i] = i!
```

and:

```
invFact[i] = inverse of i!
```

This is a very common competitive programming technique.

25. Example C++ Implementation

For competitive programming, you might eventually use something like:

```
C++
const long long MOD = 1e9 + 7;

long long power(long long a, long long b) {
    long long result = 1;

    while (b > 0) {
        if (b & 1)
            result = result * a % MOD;

        a = a * a % MOD;
        b >>= 1;
    }

    return result;
}
```

Then:

```
C++
long long nCr(long long n, long long r) {
    if (r < 0 || r > n)
        return 0;

    long long numerator = fact[n];
    long long denominator =
        invFact[r] * invFact[n-r] % MOD;

    return numerator * denominator % MOD;
}
```

For now, don't worry too much about modular inverse. Learn the basic combinatorics first.

26. How to Recognize a Combinatorics Problem

When reading a problem, look for phrases like:

Selection

```
choose k elements
select k people
pick some elements
```

Think:

$$C(n, k)$$

Arrangement

```
arrange
order
permutation
sequence
```

Think:

$$P(n, k)$$

or factorial.

Subsets

```
all subsets
some subset
choose/not choose
```

Think:

$$2^n$$

Paths

move right/down
number of ways to reach
different paths

Think:

$$C(R + D, R)$$

Counting

number of ways
how many possible
count configurations

Think:

Combinatorics may be involved.

27. A Very Important DSA Thought Process

Suppose a problem says:

Given an array of n elements, find the number of ways to choose 3 elements.

Don't immediately think about loops.

First think:

Choose 3 from n

Therefore:

$$C(n, 3)$$

Suppose:

Arrange n different elements.

Think:

Arrangement

Therefore:

$$n!$$

Suppose:

Generate every possible subset.

Think:

Each element $\rightarrow$ take / don't take

Therefore:

$$2^n$$

Suppose:

Move from $(0, 0)$ to (n, m) using only right/down.

Think:

**Total moves = $n + m$
Choose positions of right/down moves**

Therefore:

$$C(n + m, n)$$

28. Combinatorics vs Brute Force

This distinction is extremely important.

Suppose:

```
n = 20
```

Number of subsets:

$$2^{20} = 1,048,576$$

Brute force may be possible.

But:

```
n = 50
```

Then:

$$2^{50} \approx 1.12 \times 10^{15}$$

Brute force is impossible.

But maybe the problem only asks:

How many subsets are possible?

Then:

```
answer = 2^50
```

No generation required.

So:

Combinatorics often replaces enumeration with mathematical counting.

29. The Most Important Formulas for DSA

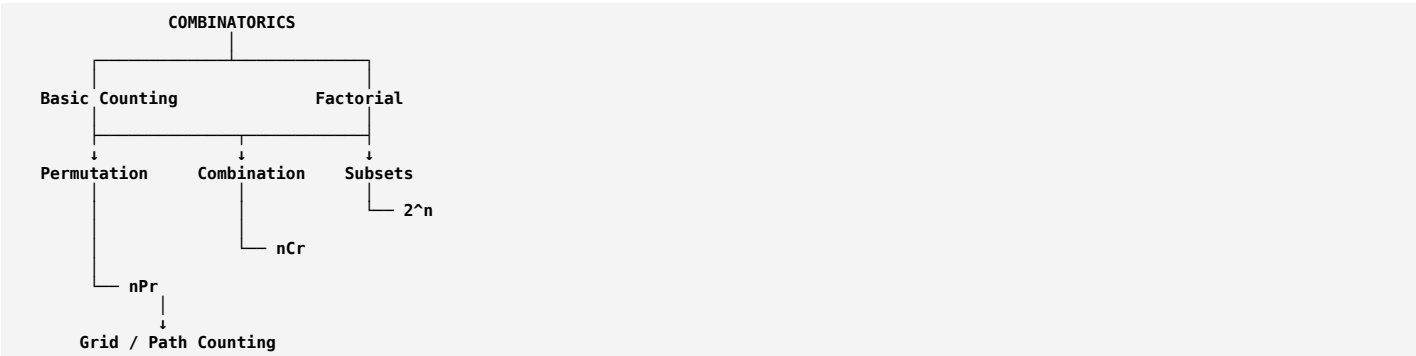
I'd recommend memorizing these first:

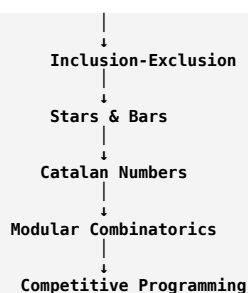
Concept	Formula
Factorial	$n!$
Permutation	$\frac{n!}{(n-r)!}$
Combination	$\frac{n!}{r!(n-r)!}$
Subsets	2^n
Grid paths	$C(R + D, R)$
Catalan	$\frac{1}{n+1} C(2n, n)$
Inclusion-Exclusion	(
Stars & Bars	$C(n + k - 1, k - 1)$

30. How I recommend you learn it for DSA

Don't study combinatorics like a pure mathematics subject.

Learn it in this order:





The key mindset

Whenever you see a DSA problem asking "**how many ways?**", don't immediately start coding.

Ask these questions:

1. Am I selecting or arranging?
2. Does order matter?
3. Can each element have independent choices?
4. Am I counting subsets?
5. Am I counting paths?
6. Am I counting duplicates/overlaps?
7. Is the search space 2^n or $n!$?
8. Is the answer too large and needs modulo?

If you become comfortable with these questions, **combinatorics starts feeling like a DSA pattern rather than difficult mathematics.**

For your current competitive-programming level, I would focus first on nCr , nPr , 2^n , **factorial**, **Pascal's identity**, **grid-path counting**, and **modular nCr** . Those will give you the biggest payoff.

If you mean the **fundamental principles of Combinatorics used in DSA**, there are a few core ideas from which most formulas come.

Think of them as the **building blocks of counting**.

1. Addition Principle — OR

If you can do something in **A** ways **OR** something else in **B** ways, and the two cases don't overlap:

$$A + B$$

Example

You can travel from Delhi to a place by:

- 3 trains
- 2 buses

Total ways:

$$3 + 2 = 5$$

DSA thinking

Whenever you see:

Case 1 OR Case 2

think:

$$+$$

2. Multiplication Principle — AND

If a process has multiple independent stages:

- First stage $\rightarrow$ **A** choices
- Second stage $\rightarrow$ **B** choices

Then:

$$A \times B$$

Example

You have:

3 shirts
2 pants

Every shirt can be combined with every pant:

$$3 \times 2 = 6$$

DSA thinking

Whenever you see:

Do this AND then this

think:



This is probably the **most fundamental counting principle**.

3. Complement Principle

Sometimes counting what you **don't want** is easier.

$$\text{Desired} = \text{Total} - \text{Undesired}$$

Example

There are 100 numbers.

Suppose 30 don't satisfy a condition.

Then numbers satisfying it:

$$100 - 30 = 70$$

DSA example

Instead of counting:

Number of arrays containing at least one 0

you can calculate:

$$\text{All arrays} - \text{arrays containing no 0}$$

This idea appears a lot in counting problems.

4. Bijection Principle

This is a more mathematical but **very powerful** principle.

If you can create a **one-to-one correspondence** between two sets, they have the same number of elements.

Suppose:

Set A:
{a, b, c}

Set B:
{1, 2, 3}

and we can map:

a → 1
b → 2
c → 3

Then:

$$|A| = |B| = 3$$

Why useful in DSA?

Sometimes the original thing is difficult to count, but you can transform it into something easier.

For example:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Number of subsets of an n element array

Instead of directly counting subsets, associate every subset with a binary string:

```
take    → 1
don't   → 0
```

For:

```
[1, 2, 3]
```

we get:

```
000
001
010
011
100
101
110
111
```

There are:

$$2^3$$

binary strings.

Therefore:

$\text{number of subsets} = 2^n$

That's essentially a bijection.

5. Division Principle

Sometimes you've counted the same thing **multiple times**.

Then divide by the number of times each thing was counted.

Example

Suppose:

```
A
B
C
```

You want to select 2 people.

If you count ordered selections:

```
AB
BA
AC
CA
BC
CB
```

There are:

$$3 \times 2 = 6$$

But each pair was counted **twice**:

```
AB
BA
```

represent the same selection.

Therefore:

$$\frac{6}{2} = 3$$

So:

$$C(3, 2) = 3$$

This principle gives us the combination formula:

$$C(n, r) = \frac{P(n, r)}{r!}$$

and therefore:

$$C(n, r) = \frac{n!}{r!(n-r)!}$$

6. Pigeonhole Principle

The basic idea:

If you put more objects into fewer boxes, at least one box must contain multiple objects.

If:

$$n + 1$$

objects are placed into:

$$n$$

boxes, then at least one box contains at least 2 objects.

Example

13 people, 12 months.

$$13 > 12$$

Therefore at least two people must have birthdays in the same month.

DSA uses

Very useful for:

- Duplicate detection
- Hashing
- Number theory
- Arrays
- Proof-based problems

7. Inclusion-Exclusion Principle

Used when different cases **overlap**.

For two sets:

$$|A \cup B| = |A| + |B| - |A \cap B|$$

Why subtract?

Because the common part was counted twice.

Example:

```
A = 10
B = 8
A ∩ B = 3
```

Then:

$$10 + 8 - 3 = 15$$

8. Principle of Double Counting

This is a very important advanced idea.

You count the **same thing in two different ways**.

Suppose you have a graph.

You can count edges by looking at:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

- edges directly
- degrees of vertices

Every edge contributes 2 to the total degree.

Therefore:

$$\sum \text{degree}(v) = 2E$$

This is a classic example of combinatorial reasoning.

It appears in:

- Graph theory
- Trees
- Counting
- Number theory
- Competitive programming

9. Recurrence Principle

Many counting problems can be broken into smaller counting problems.

For example, Fibonacci:

$$\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2)$$

Why?

Because the final step can be either:

1 step

or:

2 steps

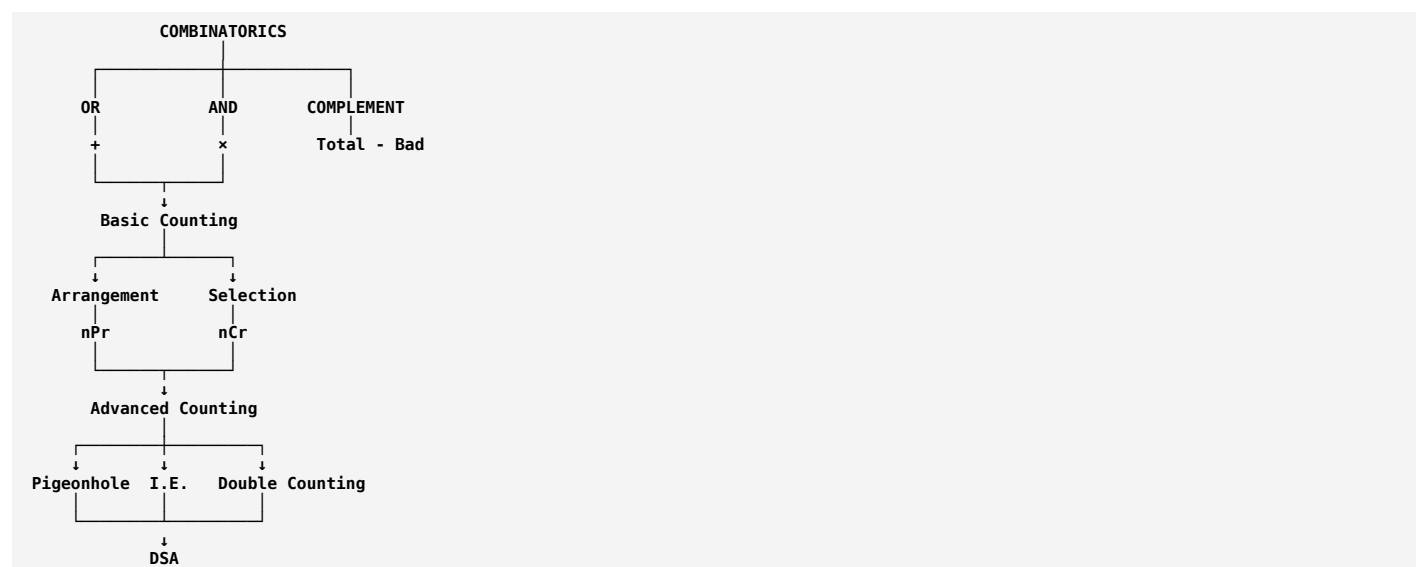
So:

$$\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2)$$

This is where combinatorics connects heavily with **Dynamic Programming**.

The Big Picture

You can remember the principles like this:



The 5 principles I'd learn first

For DSA, prioritize them in this order:

1. Addition → OR

$$A + B$$

2. Multiplication → AND

$$A \times B$$

3. Complement

$$Total - Bad$$

4. Division / avoiding overcounting

$$\frac{Total}{\text{times counted}}$$

5. Inclusion-Exclusion

$$A + B - A \cap B$$

Once these are clear, **permutations, combinations, subsets, grid paths, and most basic counting formulas become much easier to understand rather than memorize.**

Yes. The best way to understand permutations and combinations is to **derive the formulas instead of memorizing them.**

1. Permutation Formula

Permutation means:

Selecting objects where order matters.

The formula is:

$${}_n P_r = \frac{n!}{(n-r)!}$$

Let's prove it from the **multiplication principle.**

Example

Suppose we have 5 people:

A B C D E

We want to select and arrange **3 people** in 3 positions:

Position 1 → ?
Position 2 → ?
Position 3 → ?

Step 1: Fill position 1

We have 5 choices:

5

Step 2: Fill position 2

One person has already been used.

So:

4

choices remain.

Step 3: Fill position 3

Two people have already been used.

So:

3

choices remain.

Therefore, by the multiplication principle:

$$5 \times 4 \times 3 \\ = 60$$

So:

$${}^5P_3 = 60$$

General Case

Suppose we have n objects and want to arrange r of them.

The choices are:

```
Position 1 → n choices
Position 2 → n-1 choices
Position 3 → n-2 choices
...
Position r → n-r+1 choices
```

Therefore:

$${}^nP_r = n(n-1)(n-2) \cdots (n-r+1)$$

Now we want to express this using factorial.

Recall:

$$n! = n(n-1)(n-2) \cdots (n-r+1)(n-r)(n-r-1) \cdots 1$$

But we only need:

$$n(n-1)(n-2) \cdots (n-r+1)$$

So divide $n!$ by:

$$(n-r)!$$

because:

$$(n-r)! = (n-r)(n-r-1) \cdots 1$$

Therefore:

$${}^nP_r = \frac{n!}{(n-r)!}$$

That's the proof.

2. Combination Formula

Combination means:

Selecting objects where order does NOT matter.

The formula is:

$${}^nC_r = \frac{n!}{r!(n-r)!}$$

The easiest proof comes directly from permutation.

Start with permutation

Suppose:

```
A B C D
```

We want to choose 2 people.

Permutation counts:

AB
BA
AC
CA
AD
DA
BC
CB
BD
DB
CD
DC

There are:

$4P_2$

arrangements.

Calculate:

$${}^4P_2 = 4 \times 3 = 12$$

But we don't actually care about order.

For example:

AB
BA

are the **same combination**.

3. How many times is every combination counted?

Suppose we choose r elements.

Once those r elements are selected, they can be arranged in:

$$r!$$

different ways.

For example, if we select:

A B C

then permutations are:

ABC
ACB
BAC
BCA
CAB
CBA

That's:

$$3! = 6$$

So every combination is counted exactly:

$$r!$$

times when we calculate permutations.

Therefore:

$${}^nC_r = \frac{{}^nP_r}{r!}$$

Now substitute the permutation formula:

$${}^nP_r = \frac{n!}{(n-r)!}$$

Therefore:

$${}^nC_r = \frac{\frac{n!}{(n-r)!}}{r!}$$

which becomes:

$${}^nC_r = \frac{n!}{r!(n-r)!}$$

That's the proof.

4. The Difference Between Them

This is the most important intuition.

Suppose we have:

A B C

Choose 2.

Permutation

Order matters:

AB
BA
AC
CA
BC
CB

Answer:

6

Combination

Order doesn't matter:

AB
AC
BC

Answer:

3

Notice:

$$6 = 3 \times 2!$$

Therefore:

$$C(3, 2) = \frac{P(3, 2)}{2!}$$

5. Why $r!$ appears in Combination

This is something you should **understand**, not memorize.

When permutation calculates:

$$P(n, r)$$

it treats every ordering as different.

But combination says:

"I don't care about the order."

For every group of r elements, there are:

$$r!$$

different ways to arrange those same elements.

Therefore:

$$Combination = \frac{Permutation}{r!}$$

That's the entire idea.

6. Another Way to Derive Combination

There's another beautiful proof.

Suppose we have n objects and want to select r .

We can think:

Step 1

Choose the r objects:

$$C(n, r)$$

Step 2

Arrange those r objects:

$$r!$$

Therefore the number of arrangements is:

$$C(n, r) \times r!$$

But we already know the number of arrangements is:

$$P(n, r)$$

So:

$$C(n, r) \times r! = P(n, r)$$

Therefore:

$$C(n, r) = \frac{P(n, r)}{r!}$$

And:

$$C(n, r) = \frac{n!}{r!(n-r)!}$$

7. Why is $0! = 1$?

You'll see this in these formulas.

We define:

$$0! = 1$$

because factorial follows:

$$n! = n(n-1)!$$

Take $n = 1$:

$$1! = 1 \times 0!$$

Since:

$$1! = 1$$

we get:

$$1 = 1 \times 0!$$

Therefore:

$$0! = 1$$

It also makes combinatorial formulas work correctly.

For example:

$$C(n, 0) = \frac{n!}{0!n!} = 1$$

which makes sense because there is exactly **one way to choose nothing**.

8. The Most Important Intuition

Don't remember this:

"Permutation formula is $n!/(n-r)!$."

Instead remember:

Permutation

Fill positions one by one:

```
n choices
n-1 choices
n-2 choices
...
```

Therefore:

$$n(n-1)(n-2)\dots$$

Combination

First count arrangements, then remove the ordering:

$$Combination = \frac{Arrangement}{Number\ of\ arrangements\ of\ selected\ elements}$$

Therefore:

$$C(n, r) = \frac{P(n, r)}{r!}$$

and finally:

$$C(n, r) = \frac{n!}{r!(n-r)!}$$

One-line memory trick

Permutation = choose + arrange.

Combination = choose only.

Or even simpler:

```
ORDER MATTERS    → Permutation
ORDER DOESN'T    → Combination
```

This distinction is the foundation for almost every basic combinatorics problem in DSA.

Yes — this $n - r + 1$ part is confusing at first. Let's derive it **slowly**, without jumping to the formula.

We have:

$$nPr = n(n-1)(n-2)\dots$$

We want to stop after choosing **r elements**.

Example first: $n = 5, r = 3$

We have 5 objects:

A B C D E

We want to arrange 3 of them.

Position 1

5 choices:

$$5$$

Position 2

One is already used:

$$5 - 1 = 4$$

Position 3

Two are already used:

$$5 - 2 = 3$$

So:

$${}_5P_3 = 5 \times 4 \times 3$$

Notice the sequence:

$$5, 4, 3$$

Now let's see how the formula gets $n-r+1$.

General case

Suppose:

$$n = 5, \quad r = 3$$

The **first** position has:

$$n$$

choices.

The **second** position has:

$$n - 1$$

choices.

The **third** position has:

$$n - 2$$

choices.

But notice:

$$n - 2$$

can also be written as:

$$n - 3 + 1$$

because:

$$-2 = -3 + 1$$

So the last term is:

$$n - r + 1$$

because here:

$$r = 3$$

Therefore:

$$n - r + 1 = 5 - 3 + 1 = 3$$

Exactly what we got.

Why specifically +1?

This is the important part.

We start with:

```
1st choice → n
2nd choice → n-1
3rd choice → n-2
4th choice → n-3
...
```

Look at the **r-th choice**.

Before making the r-th choice, we have already selected:

$$r - 1$$

objects.

Therefore, remaining objects are:

$$n - (r - 1)$$

Now simplify:

$$\begin{aligned} n - (r - 1) \\ = n - r + 1 \end{aligned}$$

That's where the +1 comes from.

Visualize it

Suppose:

$$n = 10, \quad r = 4$$

We want 4 positions:

```
Position 1 → 10 choices
Position 2 → 9 choices
Position 3 → 8 choices
Position 4 → 7 choices
```

So:

$$10 \times 9 \times 8 \times 7$$

Now calculate the last term using the formula:

$$\begin{aligned} n - r + 1 \\ = 10 - 4 + 1 \\ = 7 \end{aligned}$$

Correct.

Now the factorial part

We have:

$${}_nP_r = n(n-1)(n-2)\cdots(n-r+1)$$

For example, with $n=10$, $r=4$:

$${}_{10}P_4 = 10 \times 9 \times 8 \times 7$$

But:

$$10!$$

contains:

$$10 \times 9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1$$

We don't want:

$$6 \times 5 \times 4 \times 3 \times 2 \times 1$$

What is that?

$$6!$$

And:

$$6 = 10 - 4$$

Therefore the unwanted part is:

$$(10 - 4)!$$

or generally:

$$(n - r)!$$

So:

$${}_nP_r = \frac{n!}{(n - r)!}$$

The whole thing in one picture

For $n = 10$, $r = 4$:

$$\begin{array}{c} 10! = \\ 10 \times 9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1 \\ \hline \begin{array}{cc} \text{WE WANT} & \text{WE DON'T WANT} \end{array} \\ \text{unwanted part} = 6! \end{array}$$

Therefore:

$$\frac{10!}{6!} = \frac{10 \times 9 \times 8 \times 7 \times 6!}{6!}$$

Cancel $6!$:

$$= 10 \times 9 \times 8 \times 7$$

And:

$$7 = 10 - 4 + 1$$

So:

$$n - r + 1$$

is simply the **number of choices remaining for the r -th position**.

Remember this:

At position r , we have already used $r-1$ objects.

Therefore:

$$\text{remaining} = n - (r - 1) = n - r + 1$$

That's the key idea.

Yes. Let's prove **combination** from the ground up, just like we did for permutation.

The main thing you need to understand is **why we divide by $r!$** .

1. What does combination mean?

Combination means:

We select r objects from n objects, and order does NOT matter.

For example, we have:

A B C

Choose 2.

The combinations are:

AB
AC
BC

Notice:

AB = BA

Because we only care about **who was selected**, not the order.

2. Let's first count using permutation

Suppose we have:

A B C D

and want to choose 2.

Instead of directly choosing, let's first **arrange** 2 objects.

For the first position:

4 choices

For the second position:

3 choices

So:

$$4 \times 3 = 12$$

Therefore:

$${}^4P_2 = 12$$

Let's actually write all of them:

AB
AC
AD

BA
BC
BD

CA
CB
CD

DA
DB
DC

There are 12.

3. But we have a problem

We don't want permutations.

We want combinations.

Look carefully:

AB
BA

These are actually the **same combination**.

Similarly:

AC
CA

same combination.

And:

AD
DA

same combination.

So permutation is **counting each combination multiple times**.

4. How many times is each combination counted?

This is the key question.

Take one combination:

ABC

Suppose we selected 3 elements.

How many different ways can these 3 elements be arranged?

ABC
ACB
BAC
BCA
CAB
CBA

There are:

$$3! = 6$$

arrangements.

Therefore, when permutation counts ABC, it actually counts:

$$3!$$

different orders of the same selection.

So every combination is counted exactly:

$$r!$$

times.

5. Therefore, how do we fix it?

Permutation contains duplicates.

If every combination was counted $r!$ times, we simply divide by $r!$.

Therefore:

$$nC_r = \frac{nPr}{r!}$$

This is the **core proof**.

6. Now substitute the permutation formula

We already proved:

$$nPr = \frac{n!}{(n-r)!}$$

So:

$$nC_r = \frac{nPr}{r!}$$

Substitute:

$$nC_r = \frac{1}{r!} \times \frac{n!}{(n-r)!}$$

Therefore:

$$nC_r = \frac{n!}{r!(n-r)!}$$

That's the combination formula.

7. Let's prove it with a real example

Take:

$$n = 5, \quad r = 3$$

We have:

A B C D E

We want to select 3.

Step 1 — Count permutations

First position:

5

Second:

4

Third:

3

Therefore:

$$5P_3 = 5 \times 4 \times 3 = 60$$

Step 2 — Remove duplicate orderings

Take one selection:

ABC

It can appear as:

ABC
ACB
BAC
BCA
CAB
CBA

That's:

$$3! = 6$$

So every group of 3 was counted 6 times.

Therefore:

$$\frac{60}{6} = 10$$

Hence:

$$5C_3 = 10$$

8. Formula

Using:

$${}_nP_r = \frac{n!}{(n-r)!}$$

we get:

$${}_nC_r = \frac{n!}{(n-r)!} \times \frac{1}{r!}$$

Therefore:

$${}_nC_r = \frac{n!}{r!(n-r)!}$$

9. But WHY exactly $r!$?

This is the most important intuition.

Suppose we select these 3 people:

A B C

The **combination** is simply:

{A, B, C}

But permutation sees:

ABC
ACB
BAC
BCA
CAB
CBA

All six are different permutations.

How many?

The first position has:

3

choices.

Second:

2

choices.

Third:

1

choice.

Therefore:

$$3 \times 2 \times 1 = 3!$$

So:

One combination produces $r!$ permutations

That's why we divide by $r!$.

10. General Proof

Let's write it mathematically.

Suppose we select r objects from n .

First:

Number of ways to arrange r objects from n :

$${}_nP_r$$

But:

Each group of r selected objects can be arranged in:

$$r!$$

ways.

Therefore:

$$nPr = nCr \times r!$$

Now divide both sides by $r!$:

$$nCr = \frac{nPr}{r!}$$

Using:

$$nPr = \frac{n!}{(n-r)!}$$

we get:

$$nCr = \frac{n!}{(n-r)!r!}$$

Therefore:

$$nCr = \frac{n!}{r!(n-r)!}$$

11. Another intuitive way to think about it

Imagine you're selecting a **team of 3** from 10 people.

You don't care about:

Alice, Bob, Charlie

versus:

Charlie, Alice, Bob

They're the same team.

But if you first calculate permutations, you're accidentally counting:

Alice Bob Charlie
 Alice Charlie Bob
 Bob Alice Charlie
 Bob Charlie Alice
 Charlie Alice Bob
 Charlie Bob Alice

as six different possibilities.

So:

Permutation
 ↓
 counts the same team 3! times
 ↓
 divide by 3!
 ↓
 Combination

12. The fundamental relationship

This is probably the **single most useful relationship to remember**:

$$nPr = nCr \times r!$$

Read it as:

Permutation = Combination × Arrangement of selected elements

Or:

$$\text{Combination} = \frac{\text{Permutation}}{\text{orderings}}$$

13. One final example

Suppose:

$$\begin{aligned} n &= 6 \\ r &= 2 \end{aligned}$$

Permutation

$${}_6P_2 = 6 \times 5 = 30$$

But each pair is counted twice:

AB
BA

because:

$$2! = 2$$

Therefore:

$$\begin{aligned} {}_6C_2 &= \frac{30}{2!} \\ &= \frac{30}{2} \\ &= \boxed{15} \end{aligned}$$

Using the formula:

$$\begin{aligned} {}_6C_2 &= \frac{6!}{2!4!} \\ &= \frac{6 \times 5}{2} \\ &= 15 \end{aligned}$$

Same answer.

The complete intuition

Don't memorize the combination formula first.

Understand this chain:

Choose r objects

↓

If order matters:

$${}_nP_r$$

↓

But we actually don't care about order.

↓

Every selection was counted:

$$r!$$

times.

↓

Remove the duplicate orderings:

$${}_nC_r = \frac{{}_nP_r}{r!}$$

↓

Substitute permutation:

$$nC_r = \frac{n!}{r!(n-r)!}$$

That's the proof. Once this clicks, nCr is no longer a formula you need to memorize—you can derive it whenever you forget it.

Yes. **Pascal's Identity** is one of the most important ideas behind combinations, and **Pascal's Triangle** is basically a visual way of applying that identity.

The key identity is:

$$C(n, r) = C(n-1, r-1) + C(n-1, r)$$

Let's understand **why this is true**, rather than memorizing it.

1. First: What is Pascal's Triangle?

It looks like this:



Each row represents combinations.

For example:

1 4 6 4 1

is:

$$C(4, 0), C(4, 1), C(4, 2), C(4, 3), C(4, 4)$$

Let's verify:

$$C(4, 0) = 1$$

$$C(4, 1) = 4$$

$$C(4, 2) = 6$$

$$C(4, 3) = 4$$

$$C(4, 4) = 1$$

So:

1 4 6 4 1

is simply the combination values for $n = 4$.

2. How do we construct the triangle?

There is a very simple rule:

Every number inside the triangle is the sum of the two numbers directly above it.

For example:

$$\begin{array}{cc} 1 & 1 \\ & \backslash / \\ & 2 \end{array}$$

Because:

$$1 + 1 = 2$$

Next:

$$\begin{array}{ccc} 1 & & 2 \\ & \backslash & / \\ & 3 & \end{array}$$

3

Because:

$$1 + 2 = 3$$

Next:

$$\begin{array}{c} 2 \quad 1 \\ \backslash \quad / \\ 3 \end{array}$$

Because:

$$2 + 1 = 3$$

So:

$$\begin{array}{c} 1 \\ 1 \ 1 \\ 1 \ 2 \ 1 \\ 1 \ 3 \ 3 \ 1 \end{array}$$

3. But WHY does this work?

This is the important part.

Suppose we want:

$$C(n, r)$$

That means:

From n people, choose r people.

Let's pick **one special person**, say Vishal.

Now there are only **two possibilities**.

Case 1: Vishal is selected

If Vishal is selected, we have already selected 1 person.

So we need:

$$r - 1$$

more people from the remaining:

$$n - 1$$

people.

Therefore:

$$C(n - 1, r - 1)$$

ways.

Case 2: Vishal is NOT selected

Then we need to choose all r people from the remaining:

$$n - 1$$

people.

Therefore:

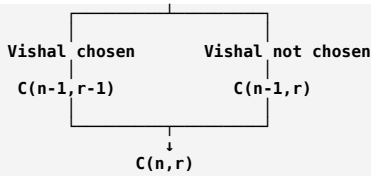
$$C(n - 1, r)$$

ways.

4. Combine the two cases

Every possible selection must belong to exactly one of these cases:

Choose r people



Therefore:

$$C(n, r) = C(n-1, r-1) + C(n-1, r)$$

This is **Pascal's Identity**.

5. Let's verify with an actual example

Suppose:

$$C(5, 2)$$

means:

Choose 2 people from 5.

Pascal's identity says:

$$C(5, 2) = C(4, 1) + C(4, 2)$$

Calculate:

$$C(4, 1) = 4$$

and:

$$C(4, 2) = 6$$

Therefore:

$$C(5, 2) = 4 + 6 = 10$$

And directly:

$$C(5, 2) = \frac{5!}{2!3!} = 10$$

Same answer.

6. Why does Pascal's Triangle have 1 on the sides?

Look at:

```

1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
  
```

The first and last number are always 1.

Why?

First number

$$C(n, 0)$$

means:

Choose zero elements from n.

There is exactly **one way**:

choose nothing

Therefore:

$$C(n, 0) = 1$$

Last number

$$C(n, n)$$

means:

Choose all n elements.

There is exactly **one way**:

choose everyone

Therefore:

$$C(n, n) = 1$$

That's why every row starts and ends with:

$$\boxed{1}$$

7. Pascal Triangle and Combinations

Now look at the rows carefully.

Row 0

1

$$C(0, 0)$$

Row 1

1 1

$$C(1, 0), C(1, 1)$$

Row 2

1 2 1

$$C(2, 0), C(2, 1), C(2, 2)$$

Row 3

1 3 3 1

$$C(3, 0), C(3, 1), C(3, 2), C(3, 3)$$

Row 4

1 4 6 4 1

$$C(4, 0), C(4, 1), C(4, 2), C(4, 3), C(4, 4)$$

So the general row is:

$$\boxed{C(n, 0), C(n, 1), C(n, 2), \dots, C(n, n)}$$

8. Another proof using the formula

We can also prove Pascal's identity algebraically.

We want to prove:

$$C(n, r) = C(n-1, r-1) + C(n-1, r)$$

Start with the right side:

$$C(n-1, r-1) + C(n-1, r)$$

Using the combination formula:

$$= \frac{(n-1)!}{(r-1)!(n-r)!} + \frac{(n-1)!}{r!(n-r-1)!}$$

Take common denominator:

$$= \frac{(n-1)!r}{r!(n-r)!} + \frac{(n-1)!(n-r)}{r!(n-r)!}$$

Combine:

$$= \frac{(n-1)![r + (n-r)]}{r!(n-r)!}$$

Since:

$$r + (n-r) = n$$

we get:

$$= \frac{n(n-1)!}{r!(n-r)!}$$

Since:

$$n(n-1)! = n!$$

we get:

$$= \frac{n!}{r!(n-r)!}$$

Therefore:

$$\boxed{C(n, r)}$$

So:

$$\boxed{C(n, r) = C(n-1, r-1) + C(n-1, r)}$$

proved.

9. Why is this useful in DSA?

This identity gives us a **DP solution for combinations**.

Suppose you want to calculate:

$$C(n, r)$$

You can create:

```
C++
dp[n][r]
```

with:

```
C++
dp[n][r] = dp[n-1][r-1] + dp[n-1][r];
```

Base cases:

```
C++
dp[n][0] = 1;
dp[n][n] = 1;
```

For example:

```
C++
long long C(int n, int r) {
    vector<vector<long long>> dp(n + 1,
                                vector<long long>(r + 1));

    for (int i = 0; i <= n; i++) {
        dp[i][0] = 1;

        for (int j = 1; j <= min(i, r); j++) {
            if (j == i)
                dp[i][j] = 1;
        }
    }
}
```



```

        else
            dp[i][j] = dp[i-1][j-1] + dp[i-1][j];
    }
}
return dp[n][r];
}

```

This is literally **Pascal's Triangle implemented as DP**.

10. One Very Important Pattern

Whenever you see:

$$C(n, r)$$

ask:

Is there a special element that I can classify as "selected" or "not selected"?

If yes, you can often derive:

$$C(n, r) = C(n-1, r-1) + C(n-1, r)$$

This same idea appears throughout combinatorial DP.

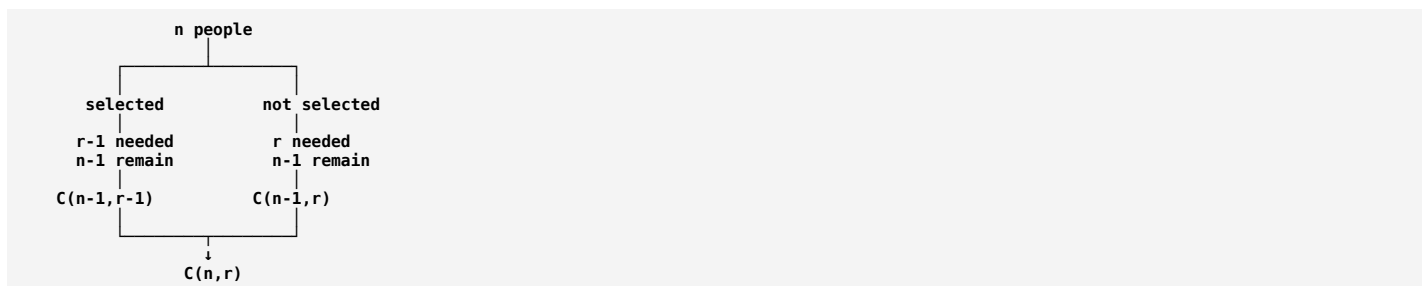
11. The easiest way to remember Pascal's Identity

Think about choosing a team.

You want:

Choose r people from n people

Pick one special person:



Therefore:

$$C(n, r) = C(n-1, r-1) + C(n-1, r)$$

And **that one identity is exactly what generates Pascal's Triangle**.

So the connection is:

$$\text{Combinations} \rightarrow \text{Pascal's Identity} \rightarrow \text{Pascal's Triangle} \rightarrow \text{DP}$$

That's why Pascal's Triangle is much more than just a mathematical triangle—it is essentially a **visual representation of the recurrence relation for nCr** .

Yes. This is a **very important competitive-programming technique**. Let's derive it from the beginning so you understand **why Fermat's Little Theorem is needed** and not just memorize the code.

1. The problem

We want to calculate:

$$\binom{n}{r} \pmod{10^9 + 7}$$

We know:

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

The problem is that **division doesn't work normally under modulo**.

For example, we cannot simply do:

```
C++
(numerator / denominator) % MOD
```

because modular arithmetic does not generally allow normal division.

We need a **modular inverse**.

2. What is modular inverse?

Suppose:

$$a = 3$$

and:

$$MOD = 7$$

We want the inverse of 3.

We need some number x such that:

$$3x \equiv 1 \pmod{7}$$

Try:

$$3 \times 5 = 15$$

and:

$$15 \bmod 7 = 1$$

Therefore:

$$3^{-1} \equiv 5 \pmod{7}$$

So instead of:

$$\frac{a}{b}$$

under modulo, we do:

$$a \times b^{-1}$$

3. Where does Fermat's Little Theorem come in?

Fermat's Little Theorem says:

If p is prime and:

$$a \not\equiv 0 \pmod{p}$$

then:

$$a^{p-1} \equiv 1 \pmod{p}$$

We can rearrange this.

Multiply both sides by:

$$a^{-1}$$

We get:

$$a^{p-2} \equiv a^{-1} \pmod{p}$$

Therefore:

$$a^{-1} \equiv a^{p-2} \pmod{p}$$

This is the key trick.

4. Apply it to $10^9 + 7$

Our modulus is:

$$MOD = 10^9 + 7$$

which is:

$$1000000007$$

and importantly:

$$1000000007 \text{ is prime}$$

Therefore, for any a not divisible by MOD :

$$a^{-1} = a^{MOD-2} \pmod{MOD}$$

So:

$$a^{-1} = a^{1000000005} \pmod{1000000007}$$

We calculate that huge power using **binary exponentiation**.

5. Apply this to nCr

We have:

$$C(n, r) = \frac{n!}{r!(n-r)!}$$

Let:

$$A = n!$$

and:

$$B = r!(n-r)!$$

Then:

$$C(n, r) = \frac{A}{B}$$

Under modulo:

$$C(n, r) = A \times B^{-1} \pmod{MOD}$$

Using Fermat:

$$B^{-1} = B^{MOD-2}$$

Therefore:

$$C(n, r) = n! \times (r!(n-r)!)^{MOD-2} \pmod{MOD}$$

That's the mathematical solution.

6. But we can do it even better

Instead of calculating:

$$(r!(n-r)!)^{MOD-2}$$

we can separately calculate the inverse factorials.

Because:

$$\frac{n!}{r!(n-r)!}$$

is:

$$n! \times (r!)^{-1} \times ((n-r)!)^{-1}$$

Therefore:

$$C(n, r) = fact[n] \times invFact[r] \times invFact[n - r] \pmod{MOD}$$

where:

$$fact[i] = i!$$

and:

$$invFact[i] = (i!)^{-1}$$

7. How do we calculate invFact?

Using Fermat.

For example:

$$invFact[r] = (r!)^{MOD-2}$$

and:

$$invFact[n - r] = ((n - r)!)^{MOD-2}$$

So we need a function for:

$$a^b \pmod{MOD}$$

8. Fast Power

We use binary exponentiation.

```
C++
long long power(long long a, long long b) {
    long long result = 1;

    while (b > 0) {
        if (b & 1) {
            result = result * a % MOD;
        }

        a = a * a % MOD;
        b >>= 1;
    }

    return result;
}
```

The important line is:

```
C++
power(a, MOD - 2)
```

which gives:

$$a^{-1} \pmod{MOD}$$

9. Complete nCr implementation

If you only need one nCr calculation:

```
C++
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll MOD = 1e9 + 7;

ll power(ll a, ll b) {
    ll result = 1;

    while (b > 0) {
        // If current bit is 1
        if (b & 1) {
            result = result * a % MOD;
        }

        // Square a
        a = a * a % MOD;
    }
}
```

```

        // Divide b by 2
        b >>= 1;
    }
    return result;
}

ll nCr(ll n, ll r) {
    if (r < 0 || r > n)
        return 0;

    // Calculate n!
    ll numerator = 1;

    for (ll i = 1; i <= n; i++) {
        numerator = numerator * i % MOD;
    }

    // Calculate r!
    ll rFact = 1;

    for (ll i = 1; i <= r; i++) {
        rFact = rFact * i % MOD;
    }

    // Calculate (n-r)!
    ll nrFact = 1;

    for (ll i = 1; i <= n-r; i++) {
        nrFact = nrFact * i % MOD;
    }

    // denominator = r! * (n-r)!
    ll denominator = rFact * nrFact % MOD;

    // denominator inverse using Fermat
    ll inverse = power(denominator, MOD - 2);

    return numerator * inverse % MOD;
}

int main() {
    cout << nCr(5, 2) << endl;

    return 0;
}

```

Output:

```
10
```

10. But there is a problem with this implementation

Suppose you have:

```
100000 queries
```

and every query asks:

```
nCr
```

The above approach repeatedly calculates factorials.

That's inefficient.

Instead, **precompute factorials once**.

11. Efficient competitive-programming version

Suppose:

$$n \leq 10^6$$

We precompute:

```
C++
fact[i] = i!
```

and:

```
C++
invFact[i] = (i!)^-1
```

Then every `nCr` query becomes:

$O(1)$

after preprocessing.

```
C++
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

const ll MOD = 1e9 + 7;

vector<ll> fact;
vector<ll> invFact;

// Fast exponentiation
ll power(ll a, ll b) {
    ll result = 1;
    while (b > 0) {
        if (b & 1) {
            result = result * a % MOD;
        }
        a = a * a % MOD;
        b >>= 1;
    }
    return result;
}

void precompute(int n) {
    fact.resize(n + 1);
    invFact.resize(n + 1);

    // factorial
    fact[0] = 1;

    for (int i = 1; i <= n; i++) {
        fact[i] = fact[i - 1] * i % MOD;
    }

    // inverse factorial of n!
    invFact[n] = power(fact[n], MOD - 2);

    // inverse factorials
    for (int i = n; i >= 1; i--) {
        invFact[i - 1] = invFact[i] * i % MOD;
    }
}

ll nCr(int n, int r) {
    if (r < 0 || r > n)
        return 0;

    return fact[n]
        * invFact[r] % MOD
        * invFact[n - r] % MOD;
}

int main() {
    precompute(1000000);

    cout << nCr(5, 2) << '\n';
    cout << nCr(10, 3) << '\n';

    return 0;
}
```

Output:

```
10
120
```

12. Why does the inverse factorial loop work?

This part is worth understanding.

We calculate:

$$\text{invFact}[n] = (n!)^{-1}$$

using:

$$\text{invFact}[n] = (n!)^{\text{MOD}-2}$$

Then:

$$\text{invFact}[n - 1]$$

can be obtained from:

$$\text{invFact}[n] \times n$$

Why?

Because:

$$(n!)^{-1} \times n$$

Since:

$$n! = n \times (n-1)!$$

we have:

$$\frac{n}{n!} = \frac{1}{(n-1)!}$$

Therefore:

$$\boxed{\text{invFact}[n-1] = \text{invFact}[n] \times n}$$

Similarly:

$$\text{invFact}[n-2] = \text{invFact}[n-1] \times (n-1)$$

and so on.

That's why:

```
C++
for (int i = n; i >= 1; i--) {
    invFact[i-1] = invFact[i] * i % MOD;
}
```

works.

13. Dry run for 5C2

We know:

$$5C2 = 10$$

Formula:

$$C(5, 2) = 5! \times (2!)^{-1} \times (3!)^{-1}$$

Calculate:

$$5! = 120$$

$$2! = 2$$

$$3! = 6$$

So:

$$120 \times 2^{-1} \times 6^{-1}$$

Modulo MOD.

Fermat tells us:

$$2^{-1} = 2^{MOD-2} \pmod{MOD}$$

and:

$$6^{-1} = 6^{MOD-2} \pmod{MOD}$$

Therefore:

$$120 \times 2^{-1} \times 6^{-1}$$

gives:

$$\boxed{10}$$

14. Why can't we simply do division?

This is a **very important point**.

We have:

$$C(n, r) = \frac{n!}{r!(n-r)!}$$

You might try:

```
C++
fact[n] / (fact[r] * fact[n-r]) % MOD
```

❌ Don't do this.

Because modulo arithmetic doesn't preserve ordinary division.

Instead:

```
division
↓
multiplication by inverse
↓
modular inverse
↓
Fermat's Little Theorem
```

So:

$$\frac{a}{b} \bmod MOD = a \times b^{-1} \bmod MOD$$

and:

$$b^{-1} = b^{MOD-2} \bmod MOD$$

15. The complete chain you should remember

This is the important part:

$$C(n, r) = \frac{n!}{r!(n-r)!}$$

↓

Division is difficult under modulo.

↓

Convert division to inverse:

$$C(n, r) = n! \times (r!)^{-1} \times ((n-r)!)^{-1}$$

↓

Fermat's Little Theorem:

$$a^{-1} = a^{MOD-2} \bmod MOD$$

↓

Precompute:

$$fact[i] = i!$$

and:

$$invFact[i] = (i!)^{-1}$$

↓

Finally:

$$C(n, r) = fact[n] \times invFact[r] \times invFact[n-r] \bmod MOD$$

Complexity

With preprocessing up to N:

$$O(N)$$

For each nCr query:

$$O(1)$$

This is the standard technique you'll see constantly in **Codeforces, AtCoder, CSES, and competitive-programming combinatorics**.

Yes — this is a very important step. The confusion is because there are **two different ways of writing the same modular inverse**.

Let's go slowly.

1. Start with the normal nCr formula

We know:

$$C(n, r) = \frac{n!}{r!(n-r)!}$$

Let's name the denominator:

$$D = r!(n-r)!$$

Then:

$$C(n, r) = \frac{n!}{D}$$

2. Division under modulo

When working modulo MOD , we cannot directly divide by D .

Instead:

$$\frac{n!}{D}$$

becomes:

$$n! \times D^{-1}$$

where:

$$D^{-1}$$

means **modular inverse of D** .

So:

$$C(n, r) = n! \times [r!(n-r)!]^{-1} \pmod{MOD}$$

3. Apply Fermat's Little Theorem

Fermat tells us:

$$a^{-1} \equiv a^{MOD-2} \pmod{MOD}$$

So if:

$$a = r!(n-r)!$$

then:

$$[r!(n-r)!]^{-1} = [r!(n-r)!]^{MOD-2}$$

Therefore:

$$C(n, r) = n! \times [r!(n-r)!]^{MOD-2} \pmod{MOD}$$

This is the expression you wrote.

4. Now where does `invFact[r]` come from?

This is the important part.

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

We have:

$$[r!(n-r)!]^{-1}$$

There is a basic mathematical property:

$$(ab)^{-1} = a^{-1}b^{-1}$$

For example:

$$(2 \times 3)^{-1} = 2^{-1} \times 3^{-1}$$

So:

$$[r!(n-r)!]^{-1}$$

can be separated:

$$= (r!)^{-1} \times ((n-r)!)^{-1}$$

Therefore:

$$C(n, r) = n! \times (r!)^{-1} \times ((n-r)!)^{-1}$$

5. Now define invFact

We define:

$$fact[i] = i!$$

Therefore:

$$fact[r] = r!$$

and:

$$fact[n-r] = (n-r)!$$

Now define:

$$invFact[i] = (i!)^{-1}$$

Therefore:

$$invFact[r] = (r!)^{-1}$$

and:

$$invFact[n-r] = ((n-r)!)^{-1}$$

So our formula becomes:

$$C(n, r) = fact[n] \times invFact[r] \times invFact[n-r]$$

Hence:

$$C(n, r) = fact[n] \times invFact[r] \times invFact[n-r] \pmod{MOD}$$

6. So are these two formulas equal?

YES.

You have:

Version 1 — Fermat directly

$$C(n, r) = n! \times [r!(n-r)!]^{MOD-2}$$

Version 2 — Precomputed inverse factorial

$$C(n, r) = fact[n] \times invFact[r] \times invFact[n-r]$$

They represent the same thing.

7. Let's prove the connection

Start with:

$$[r!(n-r)!]^{MOD-2}$$

Using:

$$(ab)^k = a^k b^k$$

we get:

$$= (r!)^{MOD-2} \times ((n-r)!)^{MOD-2}$$

Now Fermat says:

$$a^{MOD-2} = a^{-1}$$

Therefore:

$$= (r!)^{-1} \times ((n-r)!)^{-1}$$

And because:

$$\text{invFact}[r] = (r!)^{-1}$$

we get:

$$= \text{invFact}[r] \times \text{invFact}[n-r]$$

Therefore:

$$\boxed{[r!(n-r)!]^{MOD-2} = \text{invFact}[r] \times \text{invFact}[n-r]}$$

And hence:

$$\boxed{C(n, r) = \text{fact}[n] \times \text{invFact}[r] \times \text{invFact}[n-r]}$$

8. Small numerical example

Let's temporarily use a small prime:

$$MOD = 7$$

Suppose:

$$n = 4, \quad r = 2$$

Then:

$$C(4, 2) = 6$$

Formula:

$$\begin{aligned} & \frac{4!}{2!2!} \\ &= \frac{24}{4} \end{aligned}$$

Normally:

$$= 6$$

Using the first method

Denominator:

$$2! \times 2! = 4$$

We need:

$$4^{-1} \pmod{7}$$

Fermat:

$$4^{-1} = 4^{7-2} = 4^5 \pmod{7}$$

$$4^5 \pmod{7} = 2$$

because:

$$4 \times 2 = 8 \equiv 1 \pmod{7}$$

Therefore:

$$C(4, 2) = 24 \times 2 \pmod{7}$$

$$24 \pmod{7} = 3$$

$$3 \times 2 = 6$$

Correct.

Using inverse factorials

We have:

$$fact[4] = 24 \pmod{7} = 3$$

And:

$$invFact[2] = (2!)^{-1} = 2^{-1}$$

Since:

$$2 \times 4 = 8 \equiv 1 \pmod{7}$$

we have:

$$invFact[2] = 4$$

Therefore:

$$C(4, 2) = 3 \times 4 \times 4 \pmod{7}$$

$$= 48 \pmod{7}$$

$$= \boxed{6}$$

Same answer.

9. Why do we use the `invFact` version in code?

Because imagine we have:

```
n = 1,000,000
```

and thousands of queries:

```
C(100000, 50000)
C(200000, 100000)
C(500000, 200000)
...
```

We don't want to calculate:

$$(r!)^{MOD-2}$$

again and again.

Instead, calculate all:

```
C++
fact[i]
invFact[i]
```

once.

Then:

```
C++
nCr(n, r)
```

is just:

```
C++
fact[n] * invFact[r] * invFact[n-r]
```

which is:

$$O(1)$$

per query.

10. The key thing to remember

There are **three levels** here:

Level 1 — Normal mathematics

$$C(n, r) = \frac{n!}{r!(n-r)!}$$

Level 2 — Modular mathematics

Replace division with inverse:

$$C(n, r) = n! \times [r!(n-r)!]^{-1}$$

Level 3 — Fermat + precomputation

Split the inverse:

$$[r!(n-r)!]^{-1} = (r!)^{-1}((n-r)!)^{-1}$$

and call those:

$$\text{invFact}[r], \quad \text{invFact}[n-r]$$

Therefore:

$$C(n, r) = \text{fact}[n] \times \text{invFact}[r] \times \text{invFact}[n-r] \pmod{MOD}$$

So `invFact` is **not some new mathematical trick**. It's simply a convenient name for:

$$\text{invFact}[x] = (x!)^{-1} \pmod{MOD}$$

That's the whole connection.

Exactly — this is the part that usually causes confusion. **The MOD - 2 does not disappear**. It is hidden inside the definition/calculation of `invFact`.

Let's trace it carefully.

1. We start here

$$C(n, r) = n! \times [r!(n-r)!]^{MOD-2} \pmod{MOD}$$

The `MOD - 2` is there because of **Fermat's Little Theorem**:

$$a^{-1} \equiv a^{MOD-2} \pmod{MOD}$$

So:

$$[r!(n-r)!]^{MOD-2}$$

means:

Calculate the modular inverse of `r! (n-r)!`.

2. Split the expression

We can use:

$$(ab)^k = a^k b^k$$

Therefore:

$$[r!(n-r)!]^{MOD-2}$$

becomes:

$$(r!)^{MOD-2} \times ((n-r)!)^{MOD-2}$$

So our formula becomes:

$$C(n, r) = n! \times (r!)^{MOD-2} \times ((n-r)!)^{MOD-2} \pmod{MOD}$$

Notice: MOD - 2 is still here.

3. Now define invFact

We define:

$$\boxed{\text{invFact}[x] = (x!)^{MOD-2} \pmod{MOD}}$$

Why?

Because Fermat tells us:

$$(x!)^{-1} = (x!)^{MOD-2}$$

So **invFact[x]** is simply a convenient name for that huge expression.

Therefore:

$$\text{invFact}[r] = (r!)^{MOD-2}$$

and:

$$\text{invFact}[n-r] = ((n-r)!)^{MOD-2}$$

So we can replace:

$$(r!)^{MOD-2}$$

with:

$$\text{invFact}[r]$$

and:

$$((n-r)!)^{MOD-2}$$

with:

$$\text{invFact}[n-r]$$

Therefore:

$$\boxed{C(n, r) = \text{fact}[n] \times \text{invFact}[r] \times \text{invFact}[n-r] \pmod{MOD}}$$

4. So where did MOD - 2 go?

It went **inside invFact when we calculated it.**

Think of it like this:

```
invFact[r]
  ↓
(r!)^(MOD - 2)
  ↓
modular inverse of r!
```

and:

```
invFact[n-r]
  ↓
((n-r)!)^(MOD - 2)
  ↓
modular inverse of (n-r)!
```

So `MOD - 2` is **not gone**.

It has already been used to calculate `invFact`.

5. Look at the actual C++ code

This line:

```
C++
invFact[n] = power(fact[n], MOD - 2);
```

is **exactly where Fermat's `MOD - 2` is being used**.

This calculates:

$$\text{invFact}[n] = (\text{fact}[n])^{MOD-2}$$

Since:

$$\text{fact}[n] = n!$$

we get:

$$\text{invFact}[n] = (n!)^{MOD-2}$$

Then we calculate the remaining inverse factorials:

```
C++
for (int i = n; i >= 1; i--) {
    invFact[i - 1] = invFact[i] * i % MOD;
}
```

This efficiently derives:

$$\text{invFact}[n - 1], \text{invFact}[n - 2], \dots$$

from the one inverse factorial we calculated using Fermat.

6. Very important distinction

There are **two separate operations** happening.

Operation 1: Use Fermat

```
C++
invFact[n] = power(fact[n], MOD - 2);
```

This is:

$$(n!)^{MOD-2}$$

↓

$$(n!)^{-1}$$

Operation 2: Use the stored inverse

Later:

```
C++
fact[n] * invFact[r] * invFact[n-r]
```

We don't need to write `MOD - 2` again because we **already calculated the inverse**.

7. Small example

Let's use:

$$MOD = 7$$

and:

$$r = 2$$

We want:

$$\text{invFact}[2]$$

First:

$$2! = 2$$

Fermat says:

$$2^{-1} = 2^{7-2}$$

Therefore:

$$\begin{aligned} 2^{-1} &= 2^5 \pmod{7} \\ &= 32 \pmod{7} \\ &= 4 \end{aligned}$$

So:

$$\text{invFact}[2] = 4$$

Now imagine later we write:

$$\text{fact}[4] \times \text{invFact}[2] \times \text{invFact}[2]$$

We don't write:

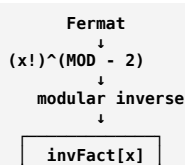
$$\text{fact}[4] \times 2^5 \times 2^5$$

because `invFact[2]` **already contains the result of** $2^5 \pmod{7}$.

That's exactly what happens with `MOD = 10^9+7`.

8. Think of `invFact` as a box

This is probably the easiest mental model:



Then when calculating `nCr`:

```

fact[n]
x
invFact[r]
x
invFact[n-r]
  
```

You are simply using the **already calculated boxes**.

Final connection

Start:

$$C(n, r) = n! \times [r!(n-r)!]^{MOD-2}$$

Split:

$$= n! \times (r!)^{MOD-2} \times ((n-r)!)^{MOD-2}$$

Define:

$$\boxed{\text{invFact}[x] = (x!)^{MOD-2}}$$

Therefore:

$$C(n, r) = fact[n] \times invFact[r] \times invFact[n - r] \pmod{MOD}$$

So the answer to "where did MOD - 2 go?" is:

It didn't disappear. We used MOD - 2 when computing invFact. After that, invFact stores the resulting modular inverse, so we don't need to write MOD - 2 again.

Sure. Here is the **standard competitive-programming code** for calculating:

$$C(n, r) \pmod{10^9 + 7}$$

using **Fermat's Little Theorem + factorial precomputation.**

```
C++
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

const ll MOD = 1e9 + 7;

// Fast Power: calculates (a^b) % MOD
ll power(ll a, ll b) {
    ll result = 1;

    while (b > 0) {
        // If b is odd
        if (b & 1) {
            result = result * a % MOD;
        }

        // Square a
        a = a * a % MOD;

        // Divide b by 2
        b >>= 1;
    }

    return result;
}

int main() {
    int N = 1000000;

    // fact[i] = i!
    vector<ll> fact(N + 1);

    // invFact[i] = (i!)^(-1) mod MOD
    vector<ll> invFact(N + 1);

    // -----
    // Step 1: Calculate factorials
    // -----

    fact[0] = 1;

    for (int i = 1; i <= N; i++) {
        fact[i] = fact[i - 1] * i % MOD;
    }

    // -----
    // Step 2: Calculate inverse of N!
    // Using Fermat:
    // (N!)^(-1) = (N!)^(MOD - 2)
    // -----

    invFact[N] = power(fact[N], MOD - 2);

    // -----
    // Step 3: Calculate all inverse
    // factorials
    // -----

    for (int i = N; i >= 1; i--) {
        invFact[i - 1] = invFact[i] * i % MOD;
    }

    // -----
    // Step 4: Calculate nCr
    // -----

    int n = 5;
    int r = 2;

    if (r > n) {
        cout << 0 << endl;
    }
    else {
        ll answer =
            fact[n]
            * invFact[r] % MOD
            * invFact[n - r] % MOD;

        cout << answer << endl;
    }
}
```

```
    return 0;
}
```

Output:

```
10
```

Because:

$$C(5, 2) = 10$$

The important part of the code

You should understand these **four pieces**:

1. Factorial

```
C++
fact[i] = fact[i - 1] * i % MOD;
```

So:

```
fact[0] = 1
fact[1] = 1
fact[2] = 2
fact[3] = 6
fact[4] = 24
fact[5] = 120
...
```

2. Fermat's Little Theorem

```
C++
invFact[N] = power(fact[N], MOD - 2);
```

This is:

$$(N!)^{-1} = (N!)^{MOD-2}$$

This is **where MOD - 2 is used**.

3. Build remaining inverse factorials

```
C++
for (int i = N; i >= 1; i--) {
    invFact[i - 1] = invFact[i] * i % MOD;
}
```

This gives:

```
invFact[0]
invFact[1]
invFact[2]
...
invFact[N]
```

where:

$$invFact[i] = (i!)^{-1}$$

4. Finally calculate nCr

```
C++
ll answer =
    fact[n]
    * invFact[r] % MOD
    * invFact[n - r] % MOD;
```

Mathematically:

$$C(n, r) = n! \times (r!)^{-1} \times ((n - r)!)^{-1}$$

or:

$$C(n, r) = fact[n] \times invFact[r] \times invFact[n - r]$$

The complete flow

```

nCr
↓
n! / (r! × (n-r)!)
↓
Division under modulo ❌
↓
Use modular inverse
↓
a-1 = a(MOD-2)
↓
Precompute fact[] and invFact[]
↓
fact[n] × invFact[r] × invFact[n-r]
↓
Answer

```

Complexity

If **N** is the maximum **n**:

- Precomputation: **O(N)**
- Each **nCr** query: **O(1)**
- Fast power: **O(log MOD)**
- Memory: **O(N)**

This is the version you should learn for **Codeforces/competitive programming**.

ChatGPT can make mistakes. Check important info.